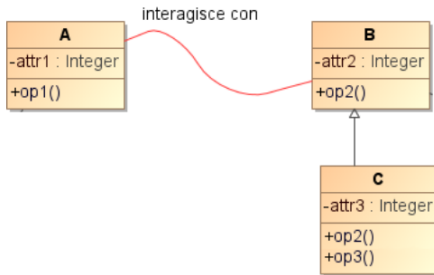
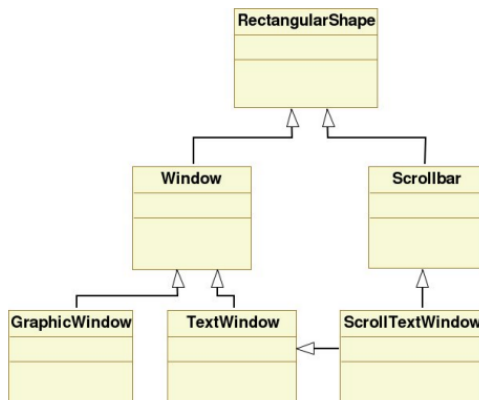


Lezione 16/10/23 ereditarietà + polimorfismo

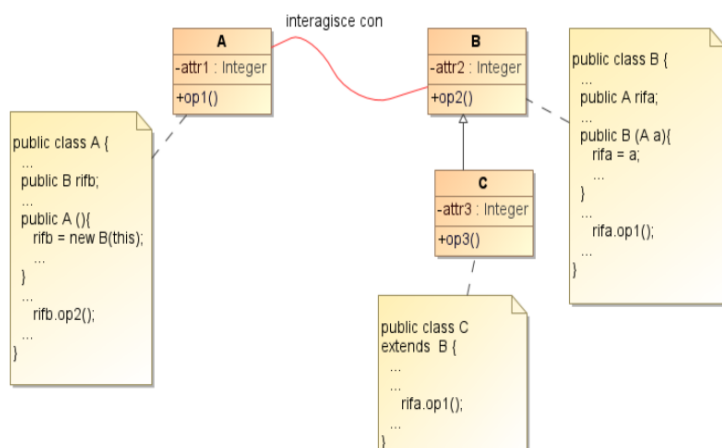
giovedì 19 ottobre 2023 11:40



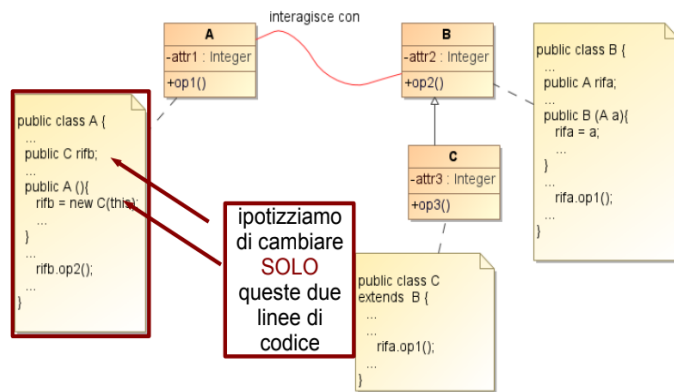
Facciamo un breve riepilogo : usiamo ancora il codice sui costruttori di altra volta . In dettaglio la classe C ha 4 caratteristiche : 2 operazioni e due attributi : operazione3() , operazione2() , attributo3 , attributo2 (attributo 2 ed operazione 2 sono ereditate dal padre). **Ricordiamo che il tipo di ritorno del costruttore è implicito : è quello della classe** . Ricordiamo inoltre che il **this** riferisce all'istanza corrente dell'oggetto, mentre il **super** è una variabile speciale che fa riferimento ad istanza corrente del padre, accedendo così alla porzione dei dati "privati del padre" : **deve essere prima istruzione nel costruttore della classe figlia** . Attenzione al seguente passo : **se dentro un costruttore invoco una new della stessa classe , nell'heap risulterà allocato spazio sia per la classe stessa (attributi nulli oppure 0), sia per una nuova classe** , mentre se nel costruttore uso il this è normale istanziazione. Nel secondo esempio si nota che c'è una differenza tra `super.proprietà` e `this.proprietà` : nel primo caso si accede al valore dell'attributo del padre , mentre nel secondo accedo ad attributo come è definito nel figlio. Posso fare `A.proprietà` (A superclasse) a posto di `super.proprietà`??? Assolutamente no in quanto il valore viene riferito in istanza e non classe. Vediamo ora un'altro concetto della programmazione o.o. : **Ereditarietà multipla** : 1 classe possa avere più superclassi (padre/madre) . In questo caso la sottoclasse eredita 1 ed 1 sola volta tutte le caratteristiche (attributi, responsabilità, vincoli ,operazioni) del padre . Non è molto raccomandata (java no) , in quanto almeno in c, il programmatore deve usarlo con responsabilità. In uml è previsto in quanto è solo un linguaggio di modellazione. Vediamo un esempio :



Dal punto di vista del modello (almeno in uml) : ovvero rappresentazione del sistema costruito con un determinato scopo , andrebbe bene , ma in realtà non va bene : non si sa l'obiettivo del sistema . I vantaggi di usare questa tecnica sono che posso creare oggetti complessi , aggregandone le funzionalità in classi diverse , quindi viene usata come usata soluzione più elegante in molti casi pratici : si aumenta la rappresentazione della realtà e si semplifica la sintassi ; come contro invece si aumenta la complessità della gestione del linguaggio e rischio elevato di name clash : si ereditano membri con lo stesso nome da più di un genitore . Come risolverlo ? O gestendo il grafo di derivazione oppure risolvere il conflitto caso per caso . Torniamo al precedente esempio :



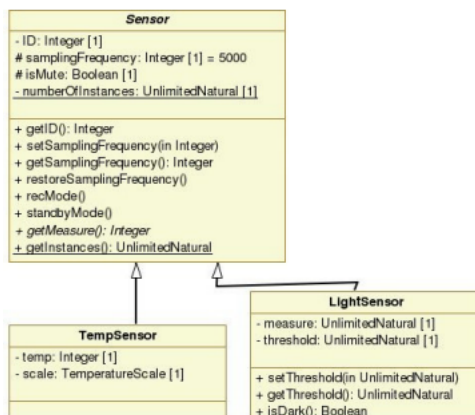
Ipotizziamo di cambiare la relazione : ovvero classe A interagisce direttamente con classe C:



Da notare che questo cambio , non porta alcun cambio rispetto ad esempio precedente, visto il concetto di generalizzazione : quindi parliamo del Principio di sostituibilità di Liskov :

- **PRINCIPIO DI SOSTITUIBILITA'** (di Liskov)
 - se $q(x)$ è una proprietà che si può dimostrare essere valida per oggetti x di tipo T , allora $q(y)$ deve essere valida per oggetti y di tipo S dove S è un sottotipo di T .
- che riformulato in termini più pragmatici:
 - data una superclasse T di S ; in tutti i contesti in cui si usa un'istanza di T , deve essere possibile utilizzare una qualsiasi istanza di S (o di una qualunque altra sottoclasse a qualsiasi livello)
- la sottoclasse deve avere la stessa semantica della superclasse

Ricordiamoci operazione, metodo ed istanza . Tornando ad esempio del sensore si ha un problema : dalla classe sensore, posso contestualizzare il sensore ? Per esempio sensore di temperatura e di luce (ereditano tutte le caratteristiche di sensor) :



Vediamolo in java :

```
public class Sensor {
    2 usages
    private int id;
    3 usages
    protected int SamplingFrequency=5000;
    2 usages
    protected boolean isMute;
    2 usages
    private static int numberOfInstances;

    public int getID() { return id; }
    public void setId(int id) { this.id = id; }
    public int getSamplingFrequency() { return SamplingFrequency; }
    public void setSamplingFrequency(int samplingFrequency) { SamplingFrequency = samplingFrequency; }
    public boolean isMute() { return isMute; }
    public void setMute(boolean mute) { isMute = mute; }
    public static int getNumberOfInstances() { return numberOfInstances; }
    public void setNumberOfInstances(int numberOfInstances) { this.numberOfInstances = numberOfInstances; }
    public void restoreSamplingFrequency() { this.SamplingFrequency=500; }
    public void recMode() { }
    public void standbyMode(){}
}
```

```

public class TempSensor extends Sensor{
    2 usages
    private int temp;
    2 usages
    private String scale;

    public int getTemp() {
        return temp;
    }

    public void setTemp(int temp) {
        this.temp = temp;
    }

    public String getScale() {
        return scale;
    }

    public void setScale(String scale) {
        this.scale = scale;
    }
}

```

```

public class LightSensor extends Sensor {
    1 usage
    private int measure;
    2 usages
    private int threshold;

    public void setMeasure(int measure) { this.measure = measure; }

    public int getThreshold() { return threshold; }

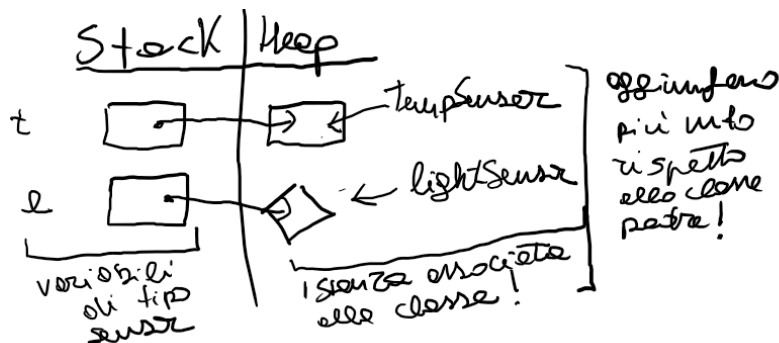
    public void setThreshold(int threshold) { this.threshold = threshold; }
    public void isBooLean(){}
}

```

```

public class Main {
    public static void main(String args[])
    {
        Sensor t=new TempSensor();
        Sensor l=new LightSensor();
        l.standbyMode();
        //l.getThreshold() not visible
        l.getMeasure();
        t.getMeasure();
        l.standbyMode();
    }
}

```



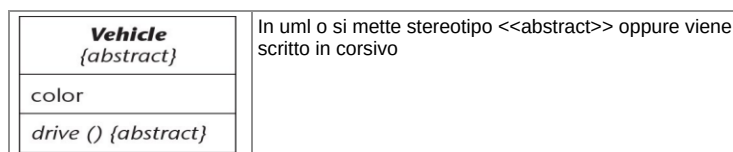
Attenzione alla riga dove c'è la allocazione dei Sensor : istanza di tipo Temp/Light sensor , mentre il tipo di variabile è di tipo Sensor : Attenzione quindi al disaccoppiamento variabile-istanza, in quanto il compilatore è molto strong-typing . Di queste variabili è possibile effettuare il cast , ma a rischio e pericolo del programmatore: l'istanza viene interpretata in altro modo :

```

((LightSensor )l).getThreshold();

```

Ma le caratteristiche / operazioni che mettiamo nella superclasse , ma dove vengono implementate ? Vengono implementate nella classe figlio , ma in modo diverso . In base a questo ragionamento andiamo a definire una variazione del concetto di classe normale : una **classe astratta** ovvero un'entità parzialmente definita : non tutto viene specificato ma lasciato in modo vago . Quindi modella un insieme di operazioni con stessa semantica ma con implementazione diversa . Almeno un'operazione viene dichiarata astratta : solo operazione!.



Non è possibile istanziarla ! . Quindi si definiscono classi figlie che vanno a complementare il comportamento di questa classe : figli hanno la responsabilità di "colmare" il gap , quindi specificano il comportamento (metodo) dell'operazione della classe astratta.

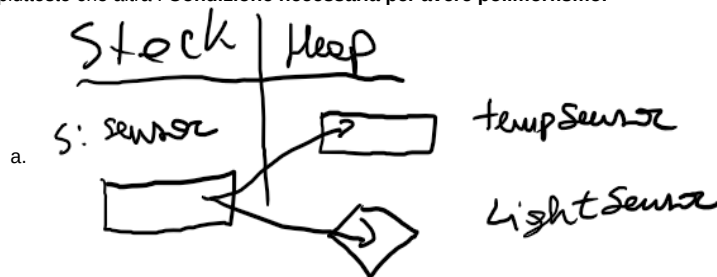
```

// Errore
Sensor s = new Sensor()

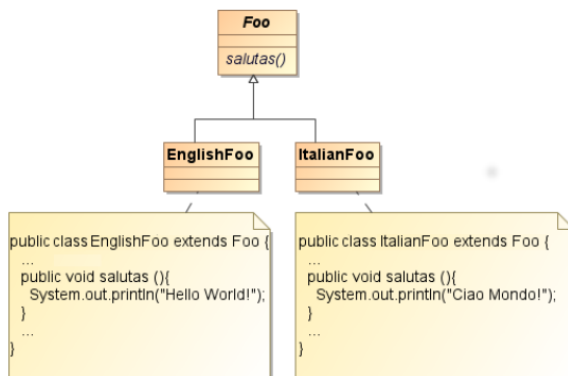
```

Quindi tutti questi concetti , portano a definire uno dei 4 cardini della progettazione o.o. : **il polimorfismo** : ovvero la capacità di una classe di comportarsi in modo differente in base al contesto di esecuzione a run-time . Per realizzare questa capacità si ha bisogno di 4 condizioni (di cui le prime tre sono impattate da programmatori e progettisti):

1. Gerarchia di classi
2. Uso variabili che hanno come tipo della superclasse
3. Meccanismo di scelta , che a seconda del contesto che crea istanza di una sottoclasse piuttosto che altra . **Condizione necessaria per avere polimorfismo.**



4. Linguaggio con supporto di binding dinamico : dipende da ambiente .
Quindi questa gestione polimorfa permette di realizzare soluzioni progettuali molto flessibili e facili da estendere (basati su generalizzazione). Possibilità di rispondere quindi a stesso messaggio in modo differente. Vediamo ora un esempio : un sistema multilingua : il sistema deve essere progettato per supportare almeno 5 lingue . Se non si usa una soluzione polimorfa , si ha una duplicazione e spreco di risorse , in quanto devo fare n sistemi diversi per minimo dettagli ; invece se soluzione polimorfa un solo sistema :



Mentre in java :

```

public class Foo {
    1 usage 2 overrides
    public void salutas() {}
}

public class ItalianFoo extends Foo {
    1 usage
    @Override
    public void salutas() {
        System.out.println("bella pee tite");
    }
}

public class EnglishFoo extends Foo {
    1 usage
    @Override
    public void salutas() {
        System.out.println("Hello world");
    }
}

public class Main {
    public static void main(String args[])
    {
        Foo f ;
        int x=0;
        if(x==0)
        {
            f=new ItalianFoo();
        }else
        {
            f=new EnglishFoo();
        }
        f.salutas();
    }
}

```

- Sarebbe possibile una gerarchia di classi astratte , ma in realtà non ha senso , in quanto non si possono istanziare . Notiamo che se ho questa istanziazione non si ha polimorfismo : il risultato è lo stesso indipendentemente dalle volte che lo eseguo.

```

Foo f1=new ItalianFoo();
f1.salutas();

```

Torniamo alla visibilità:

- a. Private : operazione/attributo solo in classe dove viene definito.

- b. Public : operazione/attributo dovunque , o meglio in tutte quelle classi che hanno relazioni con istanza di classe che la definisce.
- c. Protected : funziona come private tranne che per generalizzazione : private per tutte le atri classi , mentre public per sottoclassi e classi nello stesso pacchetto.

Vediamo ora altre due definizioni basilari : Override (necessita annotazione) e Overloading . Nel primo si parla di sovrascrittura : sovrascrivo il comportamento di un 'operazione cambiando il metodo ad essa associata : operazione ha stessa segnatura . Mentre nel secondo sovraccarico le funzionalità di una classe : aggiungendo una nuova operazione : stesso nome ma diversa segnatura .